Microsoft Agent Framework

What it is, how it's built, and how we'd use it

Why use it?

If you want to invoke anything to do with an LLM in your code, you need handling for it. At the most basic level, that means handling sending and receiving responses + waiting.

If you wanted to do anything more complex, you'd have to figure out a way to translate its text output into action.

Agent framework gives you these tools from the most basic to the hyper-specific. Things like:

- How you want the AI prompted ("you are X, respond in style Y with Z format")
- The environment specific context the AI can draw from
- Managing conversation state and history
- Calling functions (in your code) you allow it to access
- Proposing actions to take
- Among a lot of other things. Like kinda too many things.

Use an agent when...	Use a workflow when...
The task is open-ended or conversational	The process has well-defined steps
You need autonomous tool use and planning	You need explicit control over execution order
A single LLM call (possibly with tools) suffices	Multiple agents or functions must coordinate

If you can write a function to handle the task, do that instead of using an AI agent.

	Description
Agents	Individual agents that use LLMs to process inputs, call tools and MCP servers, and generate responses. Supports Microsoft Foundry, Anthropic, Azure OpenAI, OpenAI, Ollama, and more.
Harness	An opinionated agent with batteries-included capabilities for long, multi-step tasks — planning and todo tracking, context compaction, file access and memory, don't-ask-again tool approval, and observability.
Workflows	Graph-based workflows that connect agents and functions for multi-step tasks with type-safe routing, checkpointing, and human-in-the-loop support.

The framework also provides foundational building blocks, including model clients (chat completions and responses), an agent session for state management, context providers for agent memory, middleware for intercepting agent actions, and MCP clients for tool integration. Together, these components give you the flexibility and power to build interactive, robust, and safe AI applications.

How does it work at large?

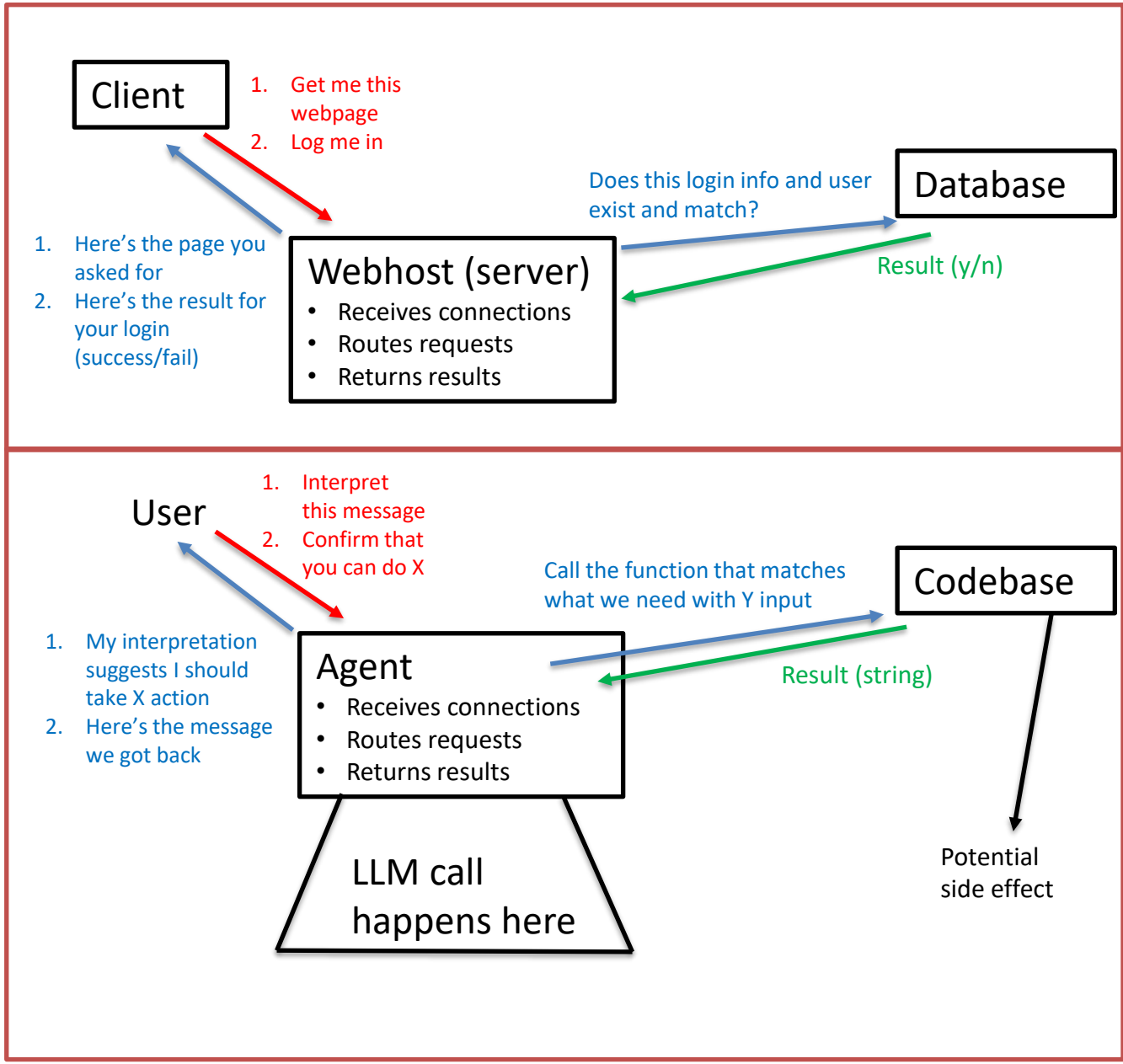
I find myself thinking of a webserver as an analogy, even if not the most well-fit.

The agent is somewhat akin to your host.

- It sits instantiated and ready for whenever you might want to access it — though unlike a real server it isn't listening; it only runs when you invoke it.
- Within that “host” (agent), there are numerous features and functionalities.
- Much like a web server, these things don’t have to necessarily actually *live* within our “host” (agent), in so much as they are accessible from there and the results get returned there.
- This framing is helpful for me just in distinguishing where exactly the actual LLM/AI models come into play, as it can be confusing at first.

The user is the client in this analogy – they send their request, confirm or deny, and wait for a response. But client is a vocab word used by the framework to represent something else, so don’t dwell on this.

→ Lab 0 · open labs/Lab0-BaseFiles/AIAgent.cs — class remarks + the RunAsync overload family.



What specifically is it built on?

An “inference service” – this is the actual LLM being utilized.

- This inference service is “hot-swappable”, both in the model itself and the way you’re accessing it.

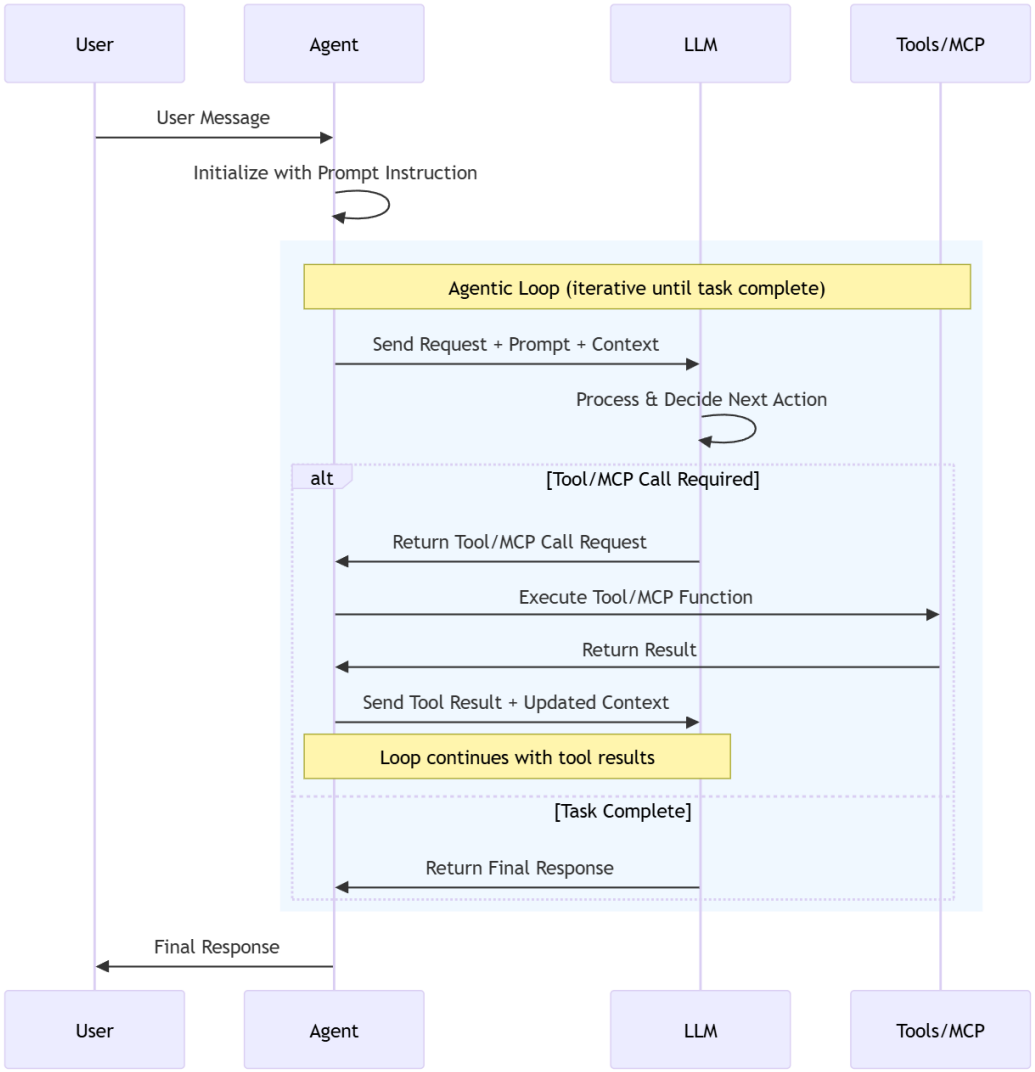
An “agent” – the programmatic logic that handles anything you might want to do with an LLM or based on the LLM’s responses.

- Runs the loop of sending messages, executing requested tools (functions), sending results back, repeating until done. Again, the agent itself is **NOT** an LLM.

A “client” – this is effectively the channel of communication that takes input + context + prompt and feeds it to the LLM before passing the result back to the agent.

An individual conversation is none of these — it lives in a session (coming up).

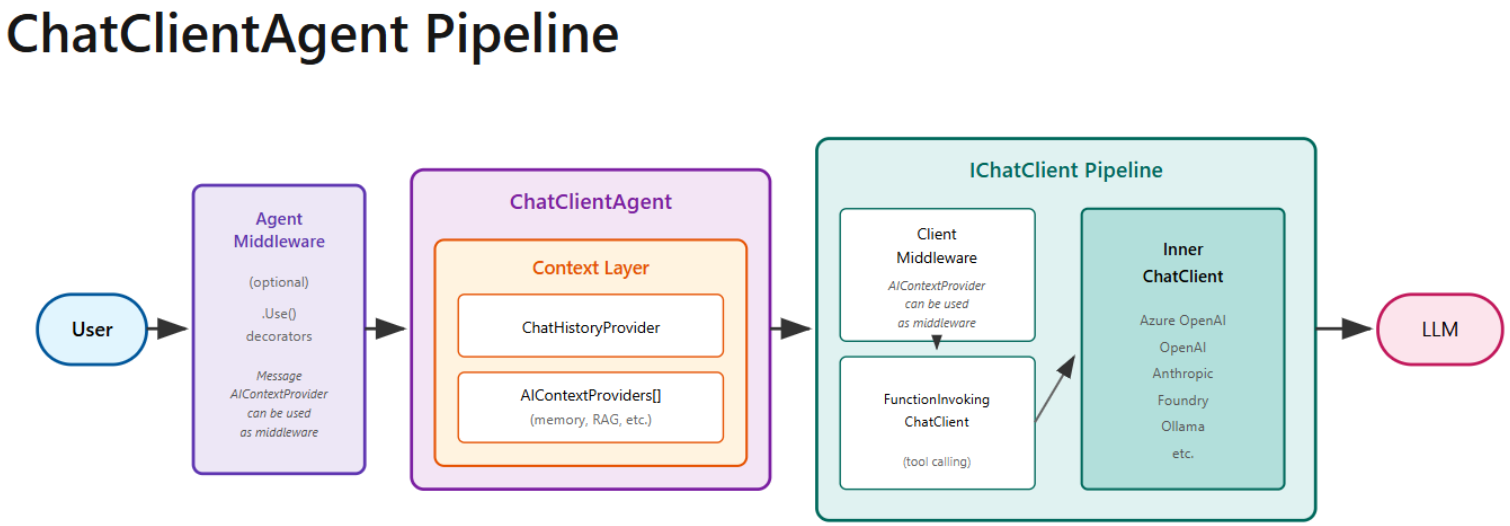
Everything builds from two central things: the base AIAgent class which creates session(s) and runs tasks, like the service contract; IChatClient (or IEmbeddingGenerator) is the contract for answering chat requests



The big picture — the ChatClientAgent pipeline

• Two contracts anchor everything: AIAgent — what an agent IS (run, sessions, serialize; the base every agent type inherits) — and IChatClient — how a model is reached (messages in, response out). ChatClientAgent INHERITS the first and COMPOSES the second.

• This picture is the whole machine. The next four slides build it from the inside out: wire client → construction line → constructor wiring → one full run.



The `ChatClientAgent` builds a pipeline with three main layers:

1. **Agent middleware** - Optional decorators that wrap the agent via `.Use()` for logging, validation, or transformation
2. **Context layer** - Manages chat history (`ChatHistoryProvider`) and injects additional context (`AIContextProviders`)
3. **Chat client layer** - The `IChatClient` with optional middleware decorators that handle LLM communication

When you call `RunAsync()`, your request flows through each layer in sequence.

→ *REFERENCE · TRACE.md — this picture as two tables (construction + run), with file + line for every hop.*

Step 1 — the wire client (a concrete IChatClient)

```
IChatClient client =
    new AzureOpenAIClient(endpoint, new AzureKeyCredential(apiKey))  // (1) the service
        .GetChatClient("gpt-4o-mini")                                // (2) the model
        .AsIChatClient();                                            // (3) the adapter

ChatResponse r = await client.GetResponseAsync(                      // (4) the raw call
    [new ChatMessage(ChatRole.User, "Any ambulances free near Lincroft?")]);
```

- (1) The Azure SDK's service client — knows WHERE (endpoint) and WHO (key), speaks Azure's REST. Provider-specific.
- (2) Narrows the service to one model deployment. Still the SDK's native type, not the neutral abstraction.
- (3) .AsIChatClient() — the adapter. From this point on the code is provider-agnostic: Ollama and Claude reach this same line through their own hops 1–2, and everything after never changes.
- (4) The raw wire call: you hand it the FULL message list (no history kept for you, no tools, no loop) and get one neutral ChatResponse back for one HTTPS round trip. This object is the innermost box of every pipeline that follows — nothing below it but the network.
- Sibling abstraction for embeddings: IEmbeddingGenerator — same idea, vectors instead of chat (RAG/memory, later).

→ REFERENCE · IChatClient.cs — GetResponseAsync / GetStreamingResponseAsync: the whole contract is two methods.

Step 2a — decoding the construction line

```
AIAgent agent =
    new AzureOpenAIClient(endpoint, credential)   // (1) the service
        .GetChatClient("gpt-4o-mini")             // (2) the model
        .AsAIAgent(options);                      // (3) adapt + build the agent
```

• (1) new AzureOpenAIClient(endpoint, credential) — the Azure SDK's service client. It knows WHERE (the endpoint) and WHO (the key) and speaks Azure's REST protocol. One per service; it can mint clients for any deployment on that endpoint. Not an IChatClient.

• (2) .GetChatClient("gpt-4o-mini") — narrows the service to ONE model deployment. Returns the SDK's native chat client — still a provider-specific type, still not the neutral abstraction.

• (3) .AsAIAgent(options) — two moves fused into one helper: adapt the native client to the neutral IChatClient, then construct a ChatClientAgent around it. Literally equivalent to .AsIChatClient() followed by new ChatClientAgent(client, options).

• The declaration shape is scope-narrowing: service → model → neutral contract → agent. You can stop at any hop — stop after .AsIChatClient() and you have step 1's raw wire client, usable without any agent.

→ If asked “why is it declared that way?” — each call narrows scope; AsAIAgent just fuses the adapt and construct hops. Ollama/Claude differ only in hops 1–2.

Step 2b — what the constructor wires up

• ChatClientAgent inherits the AIAgent contract and composes the client from step 1. Construction is where the wrapping happens:

– the ctor takes your IChatClient + ChatClientAgentOptions (ChatClientAgent.cs:118)

– wraps it with default agent middleware — including the FunctionInvokingChatClient tool loop — unless UseProvidedChatClientAsIs = true (line 131)

– defaults the history store: options?.ChatHistoryProvider ?? new InMemoryChatHistoryProvider() (line 136)

• So agent.ChatClient is already a decorated PIPELINE ending in your wire client — not the raw client you passed in.

• Streaming (RunStreamingAsync) exists and is first-class; for our headless server-side use there is little reason to use it.

```
string apiKey = Environment.GetEnvironmentVariable("AZURE_OPENAI_API_KEY")!;

AIAgent agent = new AzureOpenAIClient(
        new Uri("https://squidopenai.openai.azure.com/"),
        new AzureKeyCredential(apiKey))
    .GetChatClient("gpt-4o-mini")
    .AsAIAgent(new ChatClientAgentOptions
    {
        Name = "DispatchAgent",
        ChatOptions = new() { Instructions = "You are a city dispatch assistant." }
    });

AgentResponse response = await agent.RunAsync("Any ambulances free near Lincroft?");
Console.WriteLine(response.Text);
```

→ REFERENCE · ChatClientAgent.cs lines 118 / 131 / 136 + TRACE.md construction table.

Underlying inference service	Description	Service chat history storage supported	InMemory/Custom chat history storage supported
Microsoft Foundry Agent	An agent that uses the Foundry Agent Service as its backend.	Yes	No
Foundry Models ChatCompletion	An agent that uses any of the models deployed in the Foundry Service as its backend via ChatCompletion.	No	Yes
Foundry Models Responses	An agent that uses any of the models deployed in the Foundry Service as its backend via Responses.	Yes	Yes
Foundry Anthropic	An agent that uses a Claude model via the Foundry Anthropic Service as its backend.	No	Yes
Azure OpenAI ChatCompletion	An agent that uses the Azure OpenAI ChatCompletion service.	No	Yes
Azure OpenAI Responses	An agent that uses the Azure OpenAI Responses service.	Yes	Yes
Anthropic	An agent that uses a Claude model via the Anthropic Service as its backend.	No	Yes
OpenAI ChatCompletion	An agent that uses the OpenAI ChatCompletion service.	No	Yes
OpenAI Responses	An agent that uses the OpenAI Responses service.	Yes	Yes
Any other IChatClient	You can also use any other Microsoft.Extensions.AI.IChatClient implementation to create an agent.	Varies	Varies

Complex custom agents

It's also possible to create fully custom agents that aren't just wrappers around an `IChatClient`. The agent framework provides the `AIAgent` base type. This base type is the core abstraction for all agents, which, when subclassed, allows for complete control over the agent's behavior and capabilities.

For more information, see the documentation for Custom Agents.

Step 3 — one run, hop by hop

- This is the big-picture slide traversed left to right, one hop at a time:
- RunAsync overloads normalize your input — string, messages, or nothing (AIAgent.cs:251-334) → abstract RunCoreAsync hands off to the concrete agent (:366).
- ChatClientAgent prepares the turn: pulls history via ChatHistoryProvider.InvokingAsync (:1040), injects extra context via each AIContextProvider (:784).
- THE send is one line — chatClient.GetResponseAsync(...) (ChatClientAgent.cs:233) — one call into the decorated pipeline from step 2b.
- Inside the pipeline, FunctionInvokingChatClient loops (FunctionInvokingChatClient.cs:277): model requests a tool → your code runs → result appended → model called again — until no more requests. Only the innermost wire client ever touches the network.
- On the way out: providers are notified (:486-527), history is stored, and everything returns as AgentResponse messages → typed content items.
- Streaming is the same chain via RunStreamingAsync, yielding AgentResponseUpdate chunks instead of one AgentResponse.

→ REFERENCE · TRACE.md run table — eight hops, each with file + line.

Where state lives: sessions, messages, content

- The agent is stateless; the AgentSession is the conversation. Same session → the model sees history; no session → blank slate every call.
- One run can return several messages (tool calls, tool results, final answer). response.Text concatenates just the text; the real data is Messages → Contents.
- Content items are typed: TextContent, DataContent, UriContent, FunctionCallContent, FunctionResultContent. Non-streaming = AgentResponse.Messages; streaming = AgentResponseUpdate.Contents.

```
AgentSession session = await agent.CreateSessionAsync();

await agent.RunAsync("Report a fire at the Lincroft warehouse, high priority.",
session);

// turn 2 needs turn 1's context — "it" resolves via the session's history
AgentResponse r2 = await agent.RunAsync("Send the nearest fire unit to it.",
session);
```

Message types

Input and output from agents are represented as messages. Messages are subdivided into content items.

The Microsoft Agent Framework uses the message and content types provided by the Microsoft.Extensions.AI abstractions. Messages are represented by the `ChatMessage` class and all content classes inherit from the base `AIContent` class.

Various `AIContent` subclasses exist that are used to represent different types of content. Some are provided as part of the base Microsoft.Extensions.AI abstractions, but providers can also add their own types, where needed.

Here are some popular types from Microsoft.Extensions.AI:

Expand table

Type	Description
TextContent	Textual content that can be both input, for example, from a user or developer, and output from the agent. Typically contains the text result from an agent.
DataContent	Binary content that can be both input and output. Can be used to pass image, audio or video data to and from the agent (where supported).
UriContent	A URL that typically points at hosted content such as an image, audio or video.
FunctionCallContent	A request by an inference service to invoke a function tool.
FunctionResultContent	The result of a function tool invocation.

→ Program.cs → #region Session & REPL · AgentSession.cs: StateBag; AgentResponse.cs: Text / Usage. Run: same question with and without a session.

Context — history + injection

- Two hooks on ChatClientAgent: one ChatHistoryProvider (the conversation store) + a list of AIContextProviders (inject extra context before every model call).
- History tiers:
  - InMemoryChatHistoryProvider — the automatic default; zero setup, gone when the process exits.
  - Custom ChatHistoryProvider — persist to our own store (file, DB). The in-house end state.
  - Azure service-stored threads — persistent context with zero storage code; the conversation lives server-side. The shortcut to the harder, relevant capability while we learn.

Context layer

The context layer runs before each LLM call to build the full message history and inject additional context.

`ChatClientAgent` has two distinct provider types:

- `ChatHistoryProvider` (single) - Manages conversation history storage and retrieval
- `AIContextProviders` (list) - Injects additional context like memories, retrieved documents, or dynamic instructions

C# Copy

```
var agent = new ChatClientAgent(chatClient, new ChatClientAgentOptions
{
    ChatHistoryProvider = new InMemoryChatHistoryProvider(),
    AIContextProviders = [new MyMemoryProvider(), new MyRagProvider()],
});
```

The agent calls each provider's `InvokingAsync()` method before sending messages to the chat client with each provider's output passed as input to the next provider.

For detailed context provider patterns, see Context Providers.

```
// Injected fresh before EVERY model call — the model always sees live unit state.
sealed class UnitRosterProvider(DispatchCenter dispatch) : MessageAIContextProvider
{
    protected override ValueTask<IEnumerable<ChatMessage>> ProvideMessagesAsync(
        InvokingContext context, CancellationToken ct = default)
        => new([new ChatMessage(ChatRole.User,
            "Current units:\n" + string.Join("\n",
                dispatch.GetUnits().Select(u => $"{u.Id} {u.Type} [{u.Status}]")))]);
}

AIContextProviders = [new UnitRosterProvider(dispatch)]   // in the agent options
```

→ Program.cs → #region Context providers · hooks: ChatClientAgent.cs ChatHistoryProvider / AIContextProviders. Run the keyed lookup; RAG on demand.

Tools

• A tool is a described C# capability: AIFunctionFactory.Create(method) turns the signature + [Description] attributes into the JSON schema the model sees. The descriptions ARE the model's API docs.

• The model never executes anything — it emits a function-call request; the framework runs your code and feeds the result back into the loop.

• Tools return .NET objects; the framework handles serialization. The same slot also covers MCP tools and service-hosted tools.

The descriptions exist so that the LLM can reason about what methods actually fit the goal it's looking to accomplish.

```
[Description("Dispatch an available unit to an active incident.")]
static string DispatchUnit(
    [Description("The unit ID, e.g. POL111")] string unitId,
    [Description("The incident ID, e.g. INC-1000")] string incidentId)
    => dispatch.DisbatchUnit(unitId, incidentId)
        ? $"Unit {unitId} dispatched to {incidentId}."
        : "Dispatch failed — check unit availability.";

ChatOptions = new()
{
    Instructions = "You are a city dispatch assistant.",
    Tools = [AIFunctionFactory.Create(DispatchUnit)]
}
```

→ Program.cs → #region Tools (the five dispatch tools) · AITool.cs: Name / Description. Run the tool beat + switch_model.

Tool approval — the human gate

- By default every tool runs without approval. Gating one is a one-line wrap: new ApprovalRequiredAIFunction(fn).
- The run then pauses: instead of executing, it returns a ToolApprovalRequestContent (name + args). Your code decides, replies CreateResponse(true/false), the agent resumes on the same session.
- Enforcement is in the host, not the prompt. Deny it and the tool never runs.

```
Tools =
[
    AIFunctionFactory.Create(GetStatus),                                    // free
    new ApprovalRequiredAIFunction(AIFunctionFactory.Create(DispatchUnit)),  // gated
]

var requests = response.Messages.SelectMany(m => m.Contents)
                       .OfType<ToolApprovalRequestContent>().ToList();
while (requests.Count > 0)
{
    var replies = requests.ConvertAll(req => {
        var call = (FunctionCallContent)req.ToolCall;
        Console.Write($"Approve {call.Name}({call.Arguments})? (y/n): ");
        return new ChatMessage(ChatRole.User,
            [req.CreateResponse(Console.ReadLine()?.Trim() == "y")]);
    });
    response = await agent.RunAsync(replies, session);
    requests = response.Messages.SelectMany(m => m.Contents)
                       .OfType<ToolApprovalRequestContent>().ToList();
}
```

→ Program.cs → #region Approval loop · live: “dispatch POL111 to INC-1000”, answer n, watch nothing execute. Approved actions can trigger further gated calls.

Aside: tool approval vs middleware

• Different mechanisms. Middleware is automated in-process policy: your code decides synchronously while the run keeps executing — the model just gets the allow/refuse and continues.

• Tool approval is a suspension protocol: the run STOPS and returns early with a ToolApprovalRequestContent instead of a final answer. The decision happens outside the agent, on any timescale — a console y/n now, a button in a UI tomorrow — and you resume with CreateResponse on the same session.

• Mechanically, approval isn't middleware at all: it's a decorator on the tool itself (a wrapped AIFunction), and the suspend/resume rides the normal message-content channel.

• They compose. Middleware for what should never or always run (hard policy, no human); approval for what needs judgment. The harness wires its standing approval rules in via an agent-level decorator that manages when the suspension fires.

→ If asked "couldn't middleware do approval?" — it can block/allow automatically; the pause-and-ask-a-human round trip is what the content mechanism buys.

Middleware — three loops, three interception points

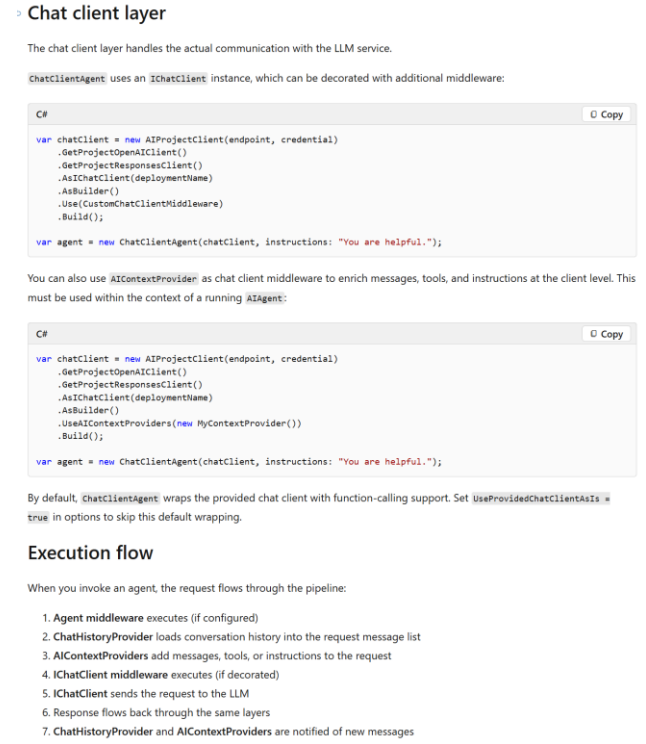
• One run = N model calls, with tool invocations between them. Each middleware type wraps one of those loops:

  – Agent-run — wraps the WHOLE run (once per RunAsync): input guardrails, audit, output filtering.

  – Function-calling — wraps EACH tool invocation: validate/rewrite/block a specific call, or terminate the loop.

  – IChatClient — wraps EACH raw model call: caching, token accounting, prompt rewriting.

• Registration is the same decorator move at each altitude: agent.AsBuilder().Use(...).Build() / chatClient.AsBuilder().Use(...).Build().



Chat client layer

The chat client layer handles the actual communication with the LLM service.

`ChatClientAgent` uses an `IChatClient` instance, which can be decorated with additional middleware:

C# Copy

```
var chatClient = new AIProjectClient(endpoint, credential)
    .GetProjectOpenAIClient()
    .GetProjectResponsesClient()
    .AsIChatClient(deploymentName)
    .AsBuilder()
    .Use(CustomChatClientMiddleware)
    .Build();

var agent = new ChatClientAgent(chatClient, instructions: "You are helpful.");
```

You can also use `AIContextProvider` as chat client middleware to enrich messages, tools, and instructions at the client level. This must be used within the context of a running `AIAgent`:

C# Copy

```
var chatClient = new AIProjectClient(endpoint, credential)
    .GetProjectOpenAIClient()
    .GetProjectResponsesClient()
    .AsIChatClient(deploymentName)
    .AsBuilder()
    .UseAIContextProviders(new MyContextProvider())
    .Build();

var agent = new ChatClientAgent(chatClient, instructions: "You are helpful.");
```

By default, `ChatClientAgent` wraps the provided chat client with function-calling support. Set `UseProvidedChatClientAsIs = true` in options to skip this default wrapping.

Execution flow

When you invoke an agent, the request flows through the pipeline:

1. **Agent middleware** executes (if configured)
2. **ChatHistoryProvider** loads conversation history into the request message list
3. **AIContextProviders** add messages, tools, or instructions to the request
4. **IChatClient middleware** executes (if decorated)
5. **IChatClient** sends the request to the LLM
6. Response flows back through the same layers
7. **ChatHistoryProvider** and **AIContextProviders** are notified of new messages

```
// agent middleware: fires once per run
agent = agent.AsBuilder().Use(AuditRun, null).Build();

// client middleware: fires once per model call inside the run
IChatClient client = azure.GetChatClient("gpt-4o-mini").AsIChatClient()
    .AsBuilder().Use(getResponseFunc: LogLlmCall,
                     getStreamingResponseFunc: null).Build();
```

→ Program.cs → #region Middleware · anchor back to ChatClientAgent.cs — the client you hand it may already be a decorated pipeline.

Structured output — and making it strict

• A model/service capability (schema-constrained decoding) the framework surfaces: RunAsync<T> derives a JSON schema from your type and hands back a typed object, not a string to parse.

• The schema guarantees the SHAPE, never the TRUTH — a required field can still be filled with a confident guess.

• Strictness is layered: required/non-nullable fields force answers; nullable fields give the model an out; an explicit “missing or uncertain” field makes it flag instead of invent. Evals and re-prompting are our application patterns on top — not framework switches.

• A shortcut worth using here: run an eval as a post-pass — Microsoft.Extensions.AI.Evaluation ships LLM-judge evaluators (and supports custom ones) that score whether the populated fields are actually grounded in the input. Catches the confident guess the schema can't.

```
[Description("A triaged incident report")]
public sealed record IncidentTriage(
    [property: Description("MedicalEmergency, Fire, CrimeInProgress, TrafficIncident")]
    string Type,
    string Priority,
    string Location,
    [property: Description("Facts NOT stated by the caller – never guess these")]
    List<string> MissingOrUncertain);

AgentResponse<IncidentTriage> resp = await agent.RunAsync<IncidentTriage>(
    "Caller reports smoke from a warehouse on Oceanport Ave, no injuries mentioned.");

IncidentTriage triage = resp.Result;   // typed object, not string parsing
```

→ Program.cs → #region Structured output · fuller: samples Agent_Step02_StructuredOutput (four variants).

The harness — everything assembled

- “Agent Harness” is a reserved term in this framework: HarnessAgent, a batteries-included agent that internally builds a ChatClientAgent and layers on: tool-loop limits, history persistence, context compaction, todo/plan providers, file memory, standing approval rules, telemetry.
- ChatClientAgent is the engine; HarnessAgent is the fully equipped vehicle built around it.
- Every battery is also available standalone — the harness is opinions, not new machinery. Read its source and you re-read this deck.

→ Lab 0 revisited · HarnessAgent.cs: the remarks block + DefaultInstructions — every item in its pipeline list is a slide we already did.

Workflows — the layer above agents

• An explicitly authored graph: executors (which can BE agents) connected by typed edges. You predefine the topology — flexibility traded for control.

• Not fully rigid: conditional routing, parallel fan-out, checkpointing, and human-in-the-loop (RequestInfoExecutor) are built in. But the flow is authored, not LLM-decided.

• Our phase 2: build workflows on top of the manual loops once those are understood — not the starting point.

```
AIAgent triage = /* classifies the incident */;
AIAgent assign = /* picks the unit to dispatch */;

var workflow = new WorkflowBuilder(triage)
    .AddEdge(triage, assign)
    .Build();

await using StreamingRun run = await InProcessExecution.RunStreamingAsync(
    workflow, new ChatMessage(ChatRole.User, "Smoke reported at Lincroft warehouse"));
```

→ Program.cs → #region Workflow (existence proof) · RequestInfoExecutor = the human gate as a first-class node. Fallback: samples/03-workflows. Point at checkpointing; don't run.

Other features — existence proofs, honest verdicts

• Backgrounding — continuation-token polling; Responses-API backends only; plain Task concurrency covers our need. Read-only.

• RAG — retrieval is literally a context provider (TextSearchProvider + vector store).

• Evaluation — Microsoft.Extensions.AI.Evaluation libraries; probably worth it, needs further consideration.

• Agent Skills — portable skill packages (SKILL.md + scripts) loaded via AgentSkillsProvider; its tools require approval by default, configurable per tool. Valuable, complex area.

• Durable Tasks — state survives restarts, resumes after failures, scales across stateless workers. Needs more study.

• CodeAct — the model writes a short program against your tools and runs it once in a sandbox, collapsing the model→tool→model loop. Approval applies to execute_code as a whole; per-operation approval means keeping those operations as direct tools. Needs vetting.

→ No lab — jump targets if questions pull: Agent_Step14 (backgrounding), AgentWithRAG_Step01, samples/03-workflows.

Difficulties — and where we land

• Feature overlap is real and by design: the composites (harness, providers) are the base pieces repackaged — the docs confirm every harness feature is also available standalone.

• So “build in-house” should mean composing the extension points — options, providers, middleware, decorators — not rebuilding the loop.

• Prompt injection is a real threat, per Microsoft's own base class: messages pass through without validation or sanitization; tool-retrieved data can carry injected instructions; all tools run unapproved by default.

  – Mitigations we already saw: approval gates on side-effecting tools, output validation, sandboxed CodeAct with narrow host tools.

• Hosted vs not: we stay not-hosted (agents run in our code). Service-stored conversation state is the one hosted piece worth borrowing.

→ Capstone · run Program.cs end-to-end; put AIAgent.cs's security remarks on screen — Microsoft says it in their own source.